

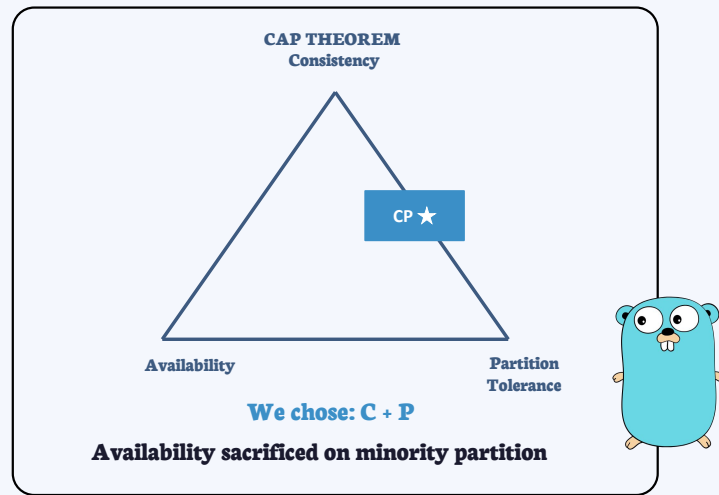
Fault-Tolerant Distributed Key-Value Store using Raft Consensus

Group 15: Aarish | Ankith | Dhvani | Ankush

Cloud VMs crash and networks partition without warning. We built a CP-compliant system that survives both without data loss or human intervention.

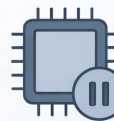
Purpose: Orchestrating Consensus within our CP System

Fault Model: Non-Byzantine



Crash-Recovery

SIGKILL via Sidecar API. Node loses memory and restarts.



Process Freeze

SIGSTOP/SIGCONT. Node alive but unable to process.



Network Partition

Bidirectional iptables DROP. Symmetric isolation.

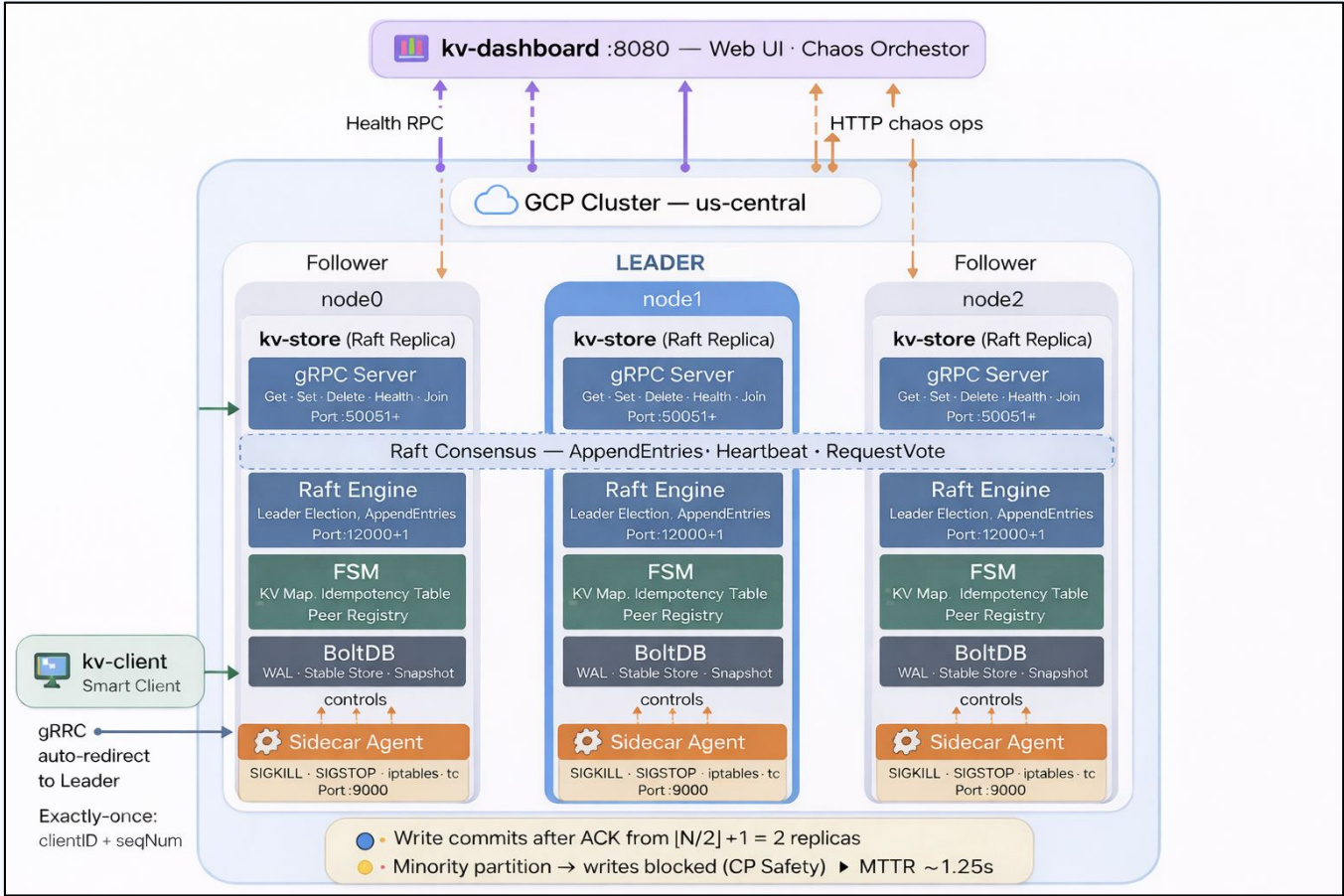


Message Delay

tc netem packet delay/loss applied to full Linux NIC.



System Functionality & Design



Replicated State Machine:

Every node runs an **identical** binary. Roles dynamically shift based on Raft elections.



Out-of-Band Fault Injection:

The **Sidecar Agent** controls the node via HTTP API without the replica's cooperation, allowing brutal chaos testing.



Strict Linearizability:

VerifyLeader() checks run before every read, ensuring deposed leaders cannot serve stale data.



Cloud Infrastructure (GCP)
N-node deployment (zones in us-central1)

Instance: e2-micro (shared-core)

CPU: 2 vCPUs

RAM: 1 GB

Disk: 10 GB Standard Boot Disk
Database: BoltDB (durable, zero-GC logs)

Network: ~1 Gbps Standard Tier





Implementation Details

Features	What it does	Proven By
DISTRIBUTED SYSTEMS PRIMITIVES		
Read-Index Follower Reads	Follower asks leader for commit_index N → waits until appliedIndex ≥ N → serves read locally. Linearizable reads without routing every GET to leader. Opt-in: --follower-read	verify_follower_read.sh 6/6
Exactly-Once Semantics	Per-client (client_id, seq_num) dedup table in FSM. Retried write after leader change is silently dropped – value is never applied twice.	I2, I3 PASS No phantom commits
Linearizable Reads (Leader)	VerifyLeader() heartbeat before every GET. Deposed leader cannot serve stale data. Minority partition → reads blocked (CP safety enforced).	P2c, L3b, N6c PASS CP safety confirmed
INFRASTRUCTURE & FAULT ENGINEERING		
Deployment Engine	Provisions N GCP VMs · cross-compiles linux/amd64 on macOS · SSH retry loop Bootstraps full N-node quorum in < 2 minutes	T1-T3: GCP 3-node 37/37 confirmed
Sidecar Agent + Fault Inject	Bidirectional iptables: INPUT DROP + OUTPUT DROP per Raft port (BUG-4 fix) NIC auto-detect: ip route get 8.8.8.8 → ens4/eth0	33/36 → 37/37 N6c, N6e PASS
Aggressive Snapshotting	SnapshotThreshold=10 entries → InstallSnapshot RPC teleports full FSM state to any lagging replica on restart. Stack: Go 1.21 · gRPC · HashiCorp Raft v1.7.3 · BoltDB	D3: 30 missed → 100% recovery ratio

Raft KV Chaos Dashboard

Healthy

Live

CLUSTER TOPOLOGY

CLUSTER HEALTH

Leader: node2
Raft Term: 3
Applied Index: 9
Nodes Alive: 3 / 3
MTTR: 0.98s

EVENT LOG

09:03:13.468 GCP node: deferring startup to node-agents (no local spawn)
02:03:38 Killing Leader: node0 (Crash failure)
09:03:38.872 KILLED node node0 via agent (GCP mode)
02:04:00 Restarting node0 - Raft log replay recovery
09:04:01.105 RESTARTED node node0 via agent (GCP mode)

CHAOS CONTROLS

Target Node: node0 (Follower)

Kill Leader

Kill Node

Restart Node

Drop Packets

Add Latency

Stop Chaos

Pause Node

Resume Node

View Node Log

REAL NETWORK CHAOS (KERNEL-LEVEL)

Partition (iptables)

Unpartition

Netem Delay

Netem Loss

Stop Netem

Reset All Chaos

KV OPERATIONS (DIRECT DASHBOARD)

Key: Value

SET

GET

DELETE

NO LOG (NONE)

Select a node and click "View Node Log"

IMPLEMENTATION PROBLEM	Linearizable Reads could lead to Leader Bottleneck	Retried Writes after Leader Change might apply twice	Netem default eth0, GCP uses ens4, breaks network delay injection
SOLUTION	Read Index enables Followers to serve Reads	Idempotent writes via dedup table in FSM	Auto-Detect NIC via ip route get 8.8.8.8, no need to hardcode





Results and Analysis



Result Category	Key Metric	Fault Method	Test Evidence	Status
WRITE LATENCY				
Quorum bypass (minority fault)	16.3 ms / 61.3 ops/s –2.5% vs baseline	2000ms netem on 1/3 follower nodes	P3 R1 – 3-node GCP	PASS
Leader bottleneck (leader fault)	1645 ms / 0.6 ops/s –99% vs baseline	500ms netem on leader + election fired	P3 R2 – 3-node GCP	PASS
Baseline (no fault)	15.9 ms / 62.9 ops/s	No fault injected	P3 R1 control 3-node GCP	PASS
LEADER RECOVERY (MTTR)				
Failover via SIGKILL	~1.25s MTTR 0→500→1250ms	kill -9 via SSH (process death)	P1 L1 – Phase 1 Liveness	PASS
Failover via iptables	~1.25s MTTR Identical to SIGKILL	Bidirectional iptables I/O DROP	P6 N6a/N6b/N6c Kernel Chaos	PASS
Isolated leader rejects writes	Write returns error (CP safety enforced)	iptables partition on leader	N6c - isolated leader can't commit	PASS
DURABILITY				
Total cluster wipe + restart	10/10 keys recovered - 100% ratio	All nodes SIGKILL'd then restarted	P4 D1	PASS
Dirty leader crash mid-write	7/7 acked keys 43 unacked = absent	Leader kill during 50-write burst	P4 D2	PASS
Snapshot catch-up (30 missed entries)	30/30 keys recovered - applied-index delta=0	Follower offline during 30 writes	P4 D3	PASS
CORRECTNESS - CP SAFETY + IDEMPOTENCY				
Exactly-once writes (dedup FSM)	Duplicate write never applied twice	Same (client_id,seq) sent 2x after failover	P5 I2/I3 Idempotency	PASS
Linearizable reads (VerifyLeader)	No stale data from deposed leader	iptables partition on reading leader	P2 P2c, P6 N6c L3b	PASS
Follower reads (Read-Index v1.3)	6/6 scenarios pass linearizable off-leader	commit_index handoff appliedIndex wait	Verify_follower_read.sh 6/6	PASS
5-node quorum generalization	43/43 at N=5 ⌊5/2⌋+1=3 quorum	3-kill → unavailable exactly at threshold	Full 6-phase suite 5-node GCP	PASS